

Introduction to MATLAB

Chapter 1 : Introduction to MATLAB programming

Julia M. Schmidt ¹

Master Quantitative Economics
Master International Affairs and Development

September 2026

¹julia.schmidt[at]dauphine.psl.eu, Website: <https://julia-m-schmidt.github.io/>

Objective

Introduce basics of MATLAB programming².

4 courses (3 hours each) :

- Introduction to MATLAB programming
- Creation of plots
- Conditional statements and loops
- Functions and optimisation methods

²These lectures are built on the material of Gauthier Vermandel for his lecture "Introduction to Finance with MATLAB". I thank him for this material and his advice. I also thank Eustache Elina, Cédric Crofils and Inès Mourelon for the elaboration of the slides and exercises.

Exam policy

3 ECTS for the “Advanced Quantitative Methods in Economics” Certificate

Evaluation: Send 4 assignments by a specific timeline to the instructor. The assignments will be started in class, and finalised at home. Special points for thinking creatively.

Each assignment ends with a short **AI log** — 2–3 lines (see next slides).

Why this course still exists

An assistant can write every line of MATLAB in this course.

What is scarce is **telling whether the output is right**.

Next year, and in a PhD, you will not write MATLAB from scratch. You will **read** it:

- a Dynare file
- a replication package
- a co-author's script
- something an assistant produced in four seconds

Someone has to open it and say *"this line is wrong"*.
That is the job. That is what this course certifies.

AI made that skill more valuable, not less.

Using AI in this course

Use whatever tools you want, on one condition:
you must be able to **explain any line you submit**, and I may ask you to.

The AI log. Each assignment ends with 2–3 lines:

- what you asked
- what it got wrong
- how you found out

Not graded. “It got nothing wrong” is a fine answer — *if* you say how you checked.

What is MATLAB ?

MATLAB (Matrix Laboratory) :

- Programming and numeric computing platform
- First version in the late 70's by Cleve Moler on Fortran
- Popularized on C in the 80's

Why MATLAB?

- Efficient at manipulating matrices and vectors
- Code easy to grasp
- Large number of toolboxes available
- Large community of MATLAB users

Installation

Software available from the University website :

- Connect to MyDauphine in English
- Go to Software
- Create a Mathworks account if you don't have one already
- Connect to the License center and activate the license with the key given by the University:
27383-79313-35234-45366-57568
- Download and install the program
- **When the installer asks which products, tick *both*: Statistics and Machine Learning Toolbox and Econometrics Toolbox.** Your licence covers everything, but the installer only installs what you tick. Check with ver.

Alternatively, you can use the online version of Matlab with your Mathworks account, using this link:
here.

Installation

Rooms and services

[How do I install and activate the MATLAB campus-wide license?](#)

Installation key: 27383-79313-35234-45366-57568

Downloading the installer

1. Create a MathWorks account using your email address (mathworks.com/accesslogin/createProfile.do).
2. Log in to the License Center at mathworks.com/licensecenter.
3. Click "Associate License" in the top-right corner.
4. When invited to do so, enter the activation key provided by your administrator.
5. Visit the following page: mathworks.com/downloads/web_downloads/select_release.
6. Click the download button to get the latest version.
7. Click the installer button to download the installer.

Students: Installation and activation

1. Locate the installer downloaded to the browser. This should be found in the default location for downloads, unless you've specified a different location. The file name for the installer will be:
 - **On Windows:** matlab_<release>_win64.exe
 - **On Mac OS X:** matlab_<release>_maci64.zipThe <version> tag corresponds to the version number.
2. Launch the installer:
 - **On Windows:** double-click the installer file downloaded in the previous step. Windows will automatically extract the installer.
 - **On Mac OS X:** double-click the installer file downloaded in the previous step. This will cause the files to be extracted and create an additional folder named "matlab_<version>_maci64", where <version> corresponds to the version number.Open this folder, and then double-click "InstallForMacOSX" to launch the installer.
3. In the MathWorks installer, select "Log in with a MathWorks account" and follow the online instructions.
4. When invited to do so, select the license you'd like to use.
5. Select the products you'd like to download and install.
6. After your products are downloaded and installed, make sure "Activate MATLAB" is checked, and then click "Next".
7. When asked to specify a username, make sure the username that appears is correct. Continue the procedure until activation is complete.

Figure: Procedure from the University website

Interface

- 4 Main windows :
 - Command window
 - Current folder
 - Workspace
 - Editor
- Prompt symbol ">>"
- Buttons

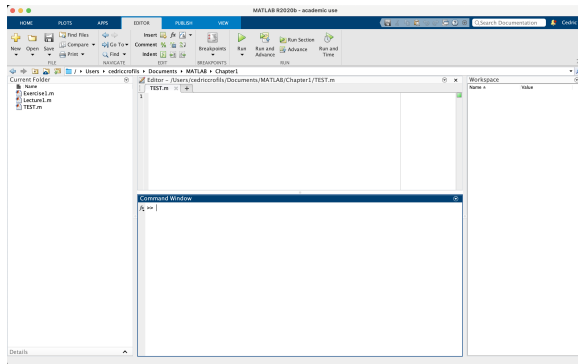


Figure: Screenshot of the MATLAB Interface

Classes

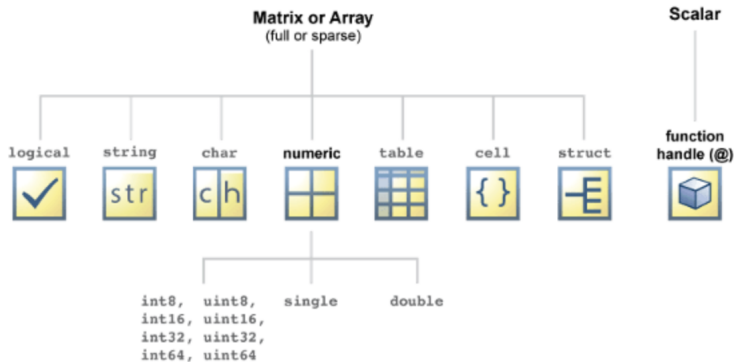


Figure: Fundamental MATLAB Classes ³

³Sources : Mathworks.com

General functions

- `clear;`
Clears the workspace
- `close all;`
Closes all the graphics
- `clc;`
Clears the command window
- `load`
Imports a .mat file into the workspace
- `save`
Saves the workspace into a .mat file
- `%`
Starts a comment
- `help`
Opens the documentation on a function
- `...`
Continues the command on the next line
- `%%`
Creates a new section
- `disp`
Prints one value to the console
- `fprintf`
Prints a formatted sentence:
`fprintf('share = %.4f\n', x)`

Arithmetic operators

- "+" Addition
- "-" Subtraction
- "*" Multiplication
- "/" Division
- "^" Powers
- "()" Brackets

```
>> 1+2*3-4/5
ans =
    6.2000
>> (1+2)*3-4/5
ans =
    8.2000
>> 5^2
ans =
    25
>> 25^(1/2)
ans =
    5
>> 25^~2
ans =
    0.0016
```

```
>> 3/7^8
ans =
    5.2040e-07
>> 8/0
ans =
    Inf
>> 0/0
ans =
    NaN
>> 27/(3*19
    27/(3*19
    ↑
Invalid expression. When calling a function or indexing a
variable, use parentheses. Otherwise, check for mismatched
delimiters.
Did you mean:
>> 27/(3*19)
```

MATLAB's elementary build-in functions

<i>Trigonometric</i>		<i>Exponential</i>	
sin	sine	exp	exponential
sinh	hyperbolic sine	log	natural logarithm
asin	inverse sine	log10	common (base10) logarithm
asinh	inverse hyperbolic sine	log2	base 2 power and scale floating point
cos	cosine	pow2	base 2 power and scale floating point
cosh	hyperbolic cosine	realpow	power that will error out on complex
acos	inverse cosine	reallog	natural logarithm of real number
acosh	inverse hyperbolic cosine	realsqrt	square root of number greater than $x^{0.5}$
tan	tangent	sqrt	square root
tanh	hyperbolic tangent	nextpow2	next higher power of 2
atan	inverse tangent	<i>Complex</i>	
atan2	four quadrant inverse	abs	absolute value
atanh	inverse hyperbolic tangent	angle	phase angle
sec	secant	complex	construct complex data from real & imaginary
sech	hyperbolic secant	conj	complex conjugate
asec	inverse secant	imag	complex imaginary part
asech	inverse hyperbolic secant	real	complex real part
csc	cosecant	unwrap	unwrap phase angle
csch	hyperbolic cosecant	isreal	true for real array
acsc	inverse cosecant	cplxpair	sort numbers into complex conjugate pairs
acsch	inverse hyperbolic cosecant	<i>Rounding and Remainder</i>	
cot	cotangent	fix	round towards zero
coth	hyperbolic cotangent	floor	round towards minus infinity
acot	inverse cotangent	ceil	round towards plus infinity
acoth	inverse hyperbolic cotangent	round	round toward nearest integer
		mod	modulus
		rem	remained after division
		sign	signum

Exercise 1

1. Create a file `exercice1.m` and edit with MATLAB to store your exercise.
2. Perform these simple calculus in MATLAB:

$$8 + 9 \quad 8 * 9 \quad \frac{8}{9} \quad 8^9$$

3. Create two variables $u = 8$ and $v = 9$ and reproduce the previous calculus using u and v .
4. Clear the workspace using `clear;` function.

5. Use build-in MATLAB function to solve:

$$\sin\left(\frac{\pi}{2}\right) \quad \sqrt{16} \quad \log(10 \cdot e^{-5})$$

Careful: $10e-5$ means 10×10^{-5} , not $10 \cdot e^{-5}$.

6. Let's consider $u = 1 + \sqrt{-4}$ Separate the imaginary and the real part.
7. Show the documentation for `sin(.)` with `help sin` (doc `sin` opens the browser).

Reminders

- Scalar : 1x1 matrix

$$\begin{bmatrix} a \end{bmatrix}$$

- Row vector : 1xn matrix

$$\begin{bmatrix} a_1 & a_2 & \dots & a_n \end{bmatrix}$$

- Column vector : nx1 matrix

$$\begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_n \end{bmatrix}$$

- Matrix : mxn matrix

$$\begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & \ddots & & \vdots \\ \vdots & & \ddots & \vdots \\ a_{m1} & \dots & \dots & a_{mn} \end{bmatrix}$$

- Diagonal matrix : nxn

$$\begin{bmatrix} a_1 & 0 & \dots & 0 \\ 0 & a_2 & & \vdots \\ \vdots & & \ddots & \vdots \\ 0 & \dots & \dots & a_n \end{bmatrix}$$

Vectors in MATLAB

- Definition of vectors and matrices : use of []
- Row vector :
- Column vector :

$$x = [1 \quad 2 \quad 3]$$

```
x = [1 2 3];  
x = 1:1:3;  
x = 1:3;  
x = zeros(1,3); x(1,1) = 1; x(1,2) =  
2; x(1,3) = 3;  
x = linspace(1,3,3);
```

$$x = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$$

```
x = [1; 2; 3];  
x = zeros(3,1); x(1,1) = 1; x(2,1) =  
2; x(3,1) = 3;  
x = [1:3]';
```

Vectors in MATLAB

```
x = [sqrt(64) pi complex(3,4)]  
iscolumn(x)  
x = transpose(x)  
length(x)  
y = [sin(pi); exp(2)]  
x + y  
y = [y; 0];  
z = 2 * x + y + 5  
z(2:end)  
z = z'
```

Exercise 2

1. Create a file `exercice2.m` and edit with MATLAB to store your exercise.
2. Create the vector $z = [1 \ 5 \ 7 \ 9]$.
3. Determine its length using the function `length(.)`.
4. Create a vector w of size 4 composed of random values using function `rand(.)`.
5. Check whether w is a row vector using function `isrow(.)`.
6. Show the 3rd value of w .
Hint: look at the documentation for the function `disp(.)`
7. Multiply the 3rd value of w by 2.

Matrices in MATLAB

Definition of vectors and matrices :

- use of []
- Other commands :
 - zeros(n,m)
 - ones(n,m)
 - eye(n, m)
 - diag(x)

Example :

$$x = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}$$

$$x = [1 \ 2; 3 \ 4; 5 \ 6];$$

$$x = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix};$$

$$x = \text{zeros}(3,2)$$

$$\begin{aligned} x(1,1) &= 1; x(1,2) = 2; x(2,1) = 3; \dots \\ x(2,2) &= 4; x(3,1) = 5; x(3,2) = 6; \end{aligned}$$

Matrices in MATLAB

Definition of random matrices :

```
A = rand(2,2)
B = randn(2,2)
```

Definition of by blocs :

```
C = [A zeros(2,2)
     2*eye(2) B]
```

Concatenation of matrices :

```
D = [A B]
D = [A ; B]
```

Operations on matrices :

```
E = A + B; E = A - B
```

Matrices in MATLAB

Matrix product :

Given a row vector W and a column vector U of length N :

$$W = [w_1, w_2, \dots, w_N], \quad U = \begin{bmatrix} u_1 \\ u_2 \\ \dots \\ u_N \end{bmatrix}$$

The inner product is defined as the linear combination:

$$W \times U = \sum_1^N w_n u_n$$

Example :

$$W = [1 \quad 2 \quad 3]$$

$$U = \begin{bmatrix} 4 \\ 5 \\ 6 \end{bmatrix}$$

$$W * U = 1 * 4 + 2 * 5 + 3 * 6 = 32$$

$$w = [1 \ 2 \ 3];$$

$$u = [4; 5; 6];$$

$$w * u$$

$$u * w$$

Matrices in MATLAB

Element-wise product (Hadamard product) :
Multiplication element by element of two
similar vectors

$$W = \begin{bmatrix} w_1 & w_2 & \dots & w_n \end{bmatrix}$$

$$U^T = \begin{bmatrix} u_1 & u_2 & \dots & u_n \end{bmatrix}$$

$$W * U^T = \begin{bmatrix} w_1 * u_1 & w_2 * u_2 & \dots & w_n * u_n \end{bmatrix}$$

$w * u'$ ← predict: does this run?
 $w. * u'$

No: w is $1 \times n$ and u' is *also* $1 \times n$, so $*$ fails. Only $.*$

works. Read the error — it names its own fix.

Note: MATLAB's `dot(w,u)` computes the *inner*
product from the previous slide (= 32), not this one.

Dot Division and Dot Power :
Similar to the element-wise product.
Example :

$$\begin{aligned} A &= \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \\ B &= A \wedge 2 \\ C &= A. \wedge 2 \end{aligned}$$

MATLAB's elementary build-in functions

zeros	Create array of all zeros	isscalar	Determine whether input is scalar
ones	Create array of all ones	isvector	Determine whether input is vector
rand	Uniformly distributed random numbers	ismatrix	Determine whether input is matrix
true	Logical 1 (true)	isrow	Determine whether input is row vector
false	Logical 0 (false)	iscolumn	Determine whether input is column vector
eye	Identity matrix	isempty	Determine whether array is empty
diag	Create diagonal matrix	sort	Sort array elements
blkdiag	Construct block diagonal matrix	sortrows	Sort rows of array, table, or timetable
cat	Concatenate arrays	issorted	Determine whether array is sorted
horzcat	Concatenate arrays horizontally	flip	Flip order of elements
vertcat	Concatenate arrays vertically	fliplr	Flip array left to right
repelem	Repeat copies of array elements	flipud	Flip array up to down
repmat	Repeat copies of array	rot90	Rotate array 90 degrees
linspace	Generate linearly spaced vector	transpose	Transpose vector or matrix
logspace	Generate logarithmically spaced vector	ctranspose	Complex conjugate transpose
freqspace	Frequency spacing for frequency response	permute	Rearrange dimensions of N-D array
meshgrid	2-D and 3-D grids	ipermute	Inverse permute dimensions of N-D array
ndgrid	Rectangular grid in N-D space	circshift	Shift array circularly
length	Length of largest array dimension	shiftdim	Shift dimensions
size	Array size	reshape	Reshape array
ndims	Number of array dimensions	squeeze	Remove singleton dimensions
numel	Number of array elements	colon	Create vectors, array subscripting, and for-loop

Table: MATLAB's Matrices and Arrays build-in functions

Matrices in MATLAB

Matrix operators

==	equal	>	strictly higher than
~=	different	<=	less or equal to
<	strictly less than	>=	greater than or equal to

Table: MATLAB's Matrices operators

ischar(x)	True if x is a character	isnan(x)	True if the elements of x are not a number
isempty(x)	True if x is empty	islogical(x)	True if element is logical
isequal(x,y)	True if the elements of x and y are identical	isnumeric(x)	True if the elements of x are numeric
isfinite(x)	True if the elements of x are finite	isreal(x)	True if the elements of x are real
isinf(x)	True if the elements of x are infinite		

Table: MATLAB's functions checking some properties of arrays/matrices

Exercise 3

Generate a matrix 5×5 full of gaussian random values with zero mean and a normalized standard deviation.

1. Determine the number of elements in the matrix above 0.

Hint: Look at the documentation for the function `sum(.)`

2. Express this number as a share of the total elements in the matrix. Store your result in variable `myshare`.

Hint: Look at the documentation for the function `numel(.)`

3. Contrast the share you obtained with the theoretical one you should obtain. Increase the number of draws in your matrix to observe the statistical convergence.

Exercise 3

4. Each time the file is run, the following message should popup in the console window: "The number of shares above zero is X " where X denotes myshare, i.e. the share of elements above zero in the matrix you have previously computed (this number should be different each iteration of the file). To do so, follow these instructions carefully:
- As a first step to illustrate concatenation, store in `str1` 'Hello ' and in `str2` 'world!', then try `[str1 str2]`. *Predict the output first. Then try it again with double quotes, "Hello " and "world!" — you get something different. Why?*
 - Now concatenate "The number of shares above zero is " with the number stored in `myshare`.
 - Employ function `disp(.)` to show a message in the console.
 - `myshare` is a number, not text. Use `num2str(.)` to convert it so the share prints correctly. *Careful: leaving out `num2str` does not give you an error — MATLAB converts silently and prints nonsense. Running is not the same as working.*
 - Now do the same with `fprintf` instead of `disp`. Which of the two would you rather read in someone else's code?

How to get draws from uniform distribution on $[0,1]$

There are two main alternatives:

- Use the function `rand`
 - Simple to use but poor coverage of space
- Use the function `haltonset`
 - More steps required but better coverage of space

True randomness does not exist on Matlab, all random sequences are pseudo random

→ Computers don't play dice

How to get "deterministic" random draws

- In simulations, it can be useful to always get the same sequence of random numbers
- In that case, you have to set a seed with any integer to the random number generator

```
rng(2);  
x = rand(1,5)
```

Running the code above will always print the following vector:

```
x = [0.4360 0.0259 0.5497 0.4353 0.4204]
```

How to get "deterministic" random draws

It is therefore possible to generate again a previous set of random numbers.

- The first step is to capture the seed s of the generator
- Then you can simulate you random number, and use the state to replicate them again.
- In that case, you have to set a seed with any integer to the random number generator

% Seed of the generator

`s = rng;`

% Generation of the first set of random numbers

`x=rand;`

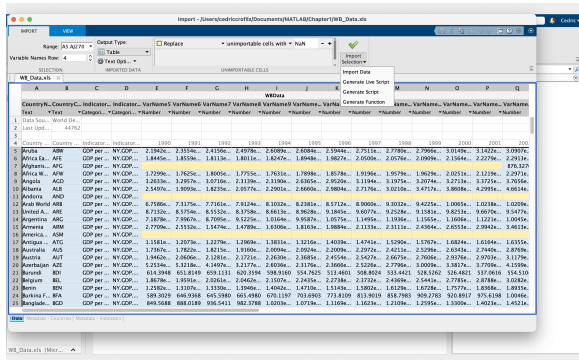
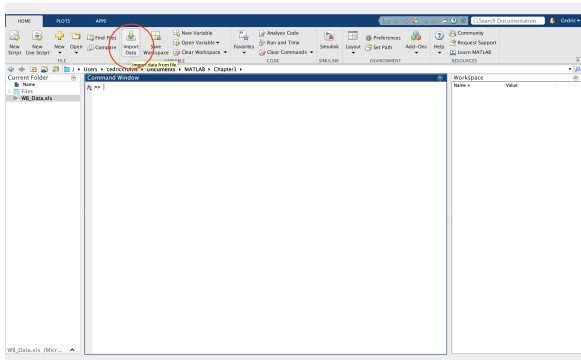
% Use of the previous seed

`rng(s);`

% Generation of the first set of random numbers

`y=rand;`

Import external files - with the assistant



Data Import and export - with the commands

Warning: Column headers from the file were modified to make them valid MATLAB identifiers before creating variable names for the table. The original column headers are saved in the VariableDescriptions property. Set 'VariableNamingRule' to 'preserve' to use the original column headers as table variable names.

```
data =
266x36 table
```

CountryName	CountryCode	IndicatorName
'Aruba'	'ABW'	'GDP per capita, PPP (current)
'Africa Eastern and Southern'	'AFE'	'GDP per capita, PPP (current)
'Afghanistan'	'AFG'	'GDP per capita, PPP (current)
'Africa Western and Central'	'AFW'	'GDP per capita, PPP (current)
'Angola'	'AGO'	'GDP per capita, PPP (current)
'Albania'	'ALB'	'GDP per capita, PPP (current)
'Andorra'	'AND'	'GDP per capita, PPP (current)
'Arab World'	'ARB'	'GDP per capita, PPP (current)
'United Arab Emirates'	'ARE'	'GDP per capita, PPP (current)
'Argentina'	'ARG'	'GDP per capita, PPP (current)
'Armenia'	'ARM'	'GDP per capita, PPP (current)
'American Samoa'	'ASM'	'GDP per capita, PPP (current)
'Antigua and Barbuda'	'ATG'	'GDP per capita, PPP (current)
'Australia'	'AUS'	'GDP per capita, PPP (current)
'Austria'	'AUT'	'GDP per capita, PPP (current)
'Azerbaijan'	'AZE'	'GDP per capita, PPP (current)
'Bahrain'	'BHT'	'GDP per capita, PPP (current)
'Belgium'	'BEL'	'GDP per capita, PPP (current)
'Benin'	'BEN'	'GDP per capita, PPP (current)
'Burkina Faso'	'BFA'	'GDP per capita, PPP (current)
'Bangladesh'	'BGD'	'GDP per capita, PPP (current)
'Bulgaria'	'BGR'	'GDP per capita, PPP (current)
'Bahrain'	'BHR'	'GDP per capita, PPP (current)
'Bhutan, The'	'BHS'	'GDP per capita, PPP (current)
'Bosnia and Herzegovina'	'BIH'	'GDP per capita, PPP (current)
'Belarus'	'BLR'	'GDP per capita, PPP (current)
'Belize'	'BLZ'	'GDP per capita, PPP (current)

For more functions see :

help iofun

Assignment

Copy paste the following commands into a new .m file:

```
% Exercice Chapter 1
% loading macro data
load Data_NelsonPlosser
% select data from 1910-1970
ourData = Data(51:end,:);
% time period vector 1910-1970
Time = dates(51:end);
% Extract data
% Stock price
SP = ourData(:,14);
% Bond Yield
BY = ourData(:,13);
% real per capita GNP
GNPPC = ourData(:,3);
```

Assignment

This command loads macroeconomic data for the US economy between 1910 to 1970. It extracts US stock prices, the bond yield rate and the real per capita GNP.

1. Compute the stock price return using the variable SP with $r_t = \log(SP_t/SP_{t-1})$. Same for the real GDP growth rate g_t .
Hint: Look for function `diff(.)`.
2. Assuming that we want to know how much asset price return r_t is affected by the GDP growth g_t . To do so, the simple regression equation is given by: $Y_i = b_0 + b_1 X_{i1} + e_i$ with $e_i \sim \mathcal{N}(0, \sigma_e^2)$. The equation above can be expressed more compactly by a set of matrices: $Y = Xb + e$.
 - (a) Create a matrix of all ones with the function `ones(.)` with the same size as g . Call it X_0 .
 - (b) Create the matrix $X = [X_0, g_t]$ composed of two vectors: one of all ones, and another with g .

Assignment

2. (c) Regression estimates of $b = [b_0, b_1]$ are typically found via least squares. The minimization problem of least squares boils down to:

$$b = (X^T X)^{-1} X^T Y$$

Compute b , display and interpret the result.

In MATLAB, write $(X' * X) \setminus (X' * Y)$ or simply $X \setminus Y$. Do **not** use `inv()`: backslash solves the system without forming the inverse — same answer, better numerics, faster. The formula describes the maths; it is not an instruction for what to type.

- (d) Confirm your result using an internal function of MATLAB performing linear regressions (use 2 methods).
- (e) Compute the residual vector e_t .
3. Now include the treasury bond yield (variable `BY` in MATLAB) in the regression model and estimate b . Are your results consistent with empirical evidence? Tips: the first value of `BY` must be removed to have the same number of observations as for g_t .

Zoom on the Halton sequence

The Halton sequence with number 2 as basis gives the following sequence:

$$s_1 = \frac{1}{2}, s_2 = s_1 - \frac{1}{4}, s_3 = s_1 + \frac{1}{4}, s_4 = s_2 - \frac{1}{8}, s_5 = s_3 - \frac{1}{8}, s_6 = s_2 + \frac{1}{8}, s_7 = s_3 + \frac{1}{8} \dots$$

$$\dots s_8 = s_4 - \frac{1}{24}, s_9 = s_5 - \frac{1}{24}, s_{10} = s_6 - \frac{1}{24}, s_{11} = s_7 - \frac{1}{24}, s_{12} = s_4 + \frac{1}{24} \dots$$

Question: What is s_{12} ?

The Halton sequence is not pseudo-random but quasi-random

→ The Halton sequence covers all the space uniformly but how to scramble it?

Zoom on the Halton sequence

```
p = haltonset(1,'Skip',1e3,'Leap',1e6);  
p = scramble(p,'RR2');  
temp = net(p,100);
```

1. Generate a 1-dimensional Halton point set (build the halton sequence with number 2 as basis; a 2-dimensional Halton point set would yield both the sequence with number 2 and number 3 as basis)
 - The 'Skip' option skips the first 1000 numbers in the sequence
 - The 'Leap' option then only retains one number every 1 000 001 point
2. The scramble function scrambles all the numbers
3. The net function extracts the first 100 numbers from our sequence

Halton sequence vs random number generator

Compare the performance of rand vs haltonset:

```
N = 100;  
p = haltonset(1,'Skip',1e3,'Leap',1e6);  
p = scramble(p,'RR2');  
temp = net(p,N);  
err_halton = abs(mean(temp)-0.5);  
err_rand = abs(mean(rand(1,N))-0.5);
```

From uniform distribution to normal distribution

Using the Halton sequence is a better alternative than rand. It can also be used as a better substitute to randn. How to do that?

If you have a sample of random draws from an uniform distribution, you can transform it into a sample of random draw from a centered-normal distribution by applying the inverse of the CDF of the normal distribution.

```
a = norminv(temp);
```

Probability Integral Transform

For X a continuous random variable, F_X its CDF, then the random variable $Y = F_X(X)$ has a standard uniform distribution